

APLIKASI PEMESANAN MAKANAN SEDERHANA

Dosen Pengampu:

Liza Afriyanti, M.Kom



Oleh:

- 1. Elviana Vania (12550124181)**
- 2. Alya Naditha (12550125198)**
- 3. Diva Septiyanti (12550125251)**

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI SULTAN SYARIF KASIM RIAU

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi di era modern sekarang ini memberikan kemudahan dalam berbagai aspek kehidupan, termasuk dalam proses pemesanan makanan. Pada umumnya, proses pemesanan makanan yang dilakukan secara manual sering kali menimbulkan berbagai kendala, seperti kesalahan pencatatan pesanan, waktu pelayanan yang lebih lama, dan kesulitan dalam pengelolaan data transaksi. Permasalahan tersebut diperlukan sebuah sistem untuk membantu proses pemesanan makanan secara lebih efektif dan efisien. Salah satu solusi yang dapat diterapkan adalah membangun aplikasi pemesanan makanan sederhana menggunakan bahasa pemrograman Python.

Aplikasi ini dirancang untuk membantu pengguna dalam melihat daftar menu dalam memilih menu makanan, menghitung total harga pesanan, serta menghasilkan informasi transaksi secara otomatis. Selain sebagai solusi sederhana untuk proses pemesanan makanan, proyek ini juga menjadi sarana penerapan konsep OOP seperti class, inheritance, abstraction, encapsulation, dan polymorphism dalam pengembangan perangkat lunak. Melalui pengembangan aplikasi ini dapat menjadikan sarana pembelajaran dalam menerapkan analisis kebutuhan, perancangan UML, implementasi program menggunakan Python, pengujian sistem, dan evaluasi prinsip SOLID pada proyek.

1.2 Rumusan Masalah

Adapun rumusan masalah dari proyek ini adalah:

1. Bagaimana merancang desain sistem aplikasi pemesanan makanan menggunakan diagram UML?
2. Bagaimana mengimplementasikan aplikasi pemesanan makanan sederhana dengan konsep Object-Oriented Programming (OOP)?
3. Bagaimana melakukan pengujian untuk memastikan setiap fitur aplikasi berjalan sesuai dengan kebutuhan pengguna?
4. Bagaimana menerapkan dan mengevaluasi prinsip-prinsip SOLID pada aplikasi pemesanan makanan yang dikembangkan?

1.3 Tujuan

Adapun tujuan dari proyek ini adalah:

1. Untuk mengetahui perancangan desain sistem aplikasi pemesanan makanan menggunakan diagram UML.
2. Untuk mengetahui pengimplementasian aplikasi pemesanan makanan sederhana dengan konsep Object-Oriented Programming (OOP).
3. Untuk mengetahui setiap fitur aplikasi berjalan sesuai dengan kebutuhan pengguna.
4. Untuk mengetahui prinsip-prinsip SOLID pada aplikasi pemesanan makanan yang dikembangkan.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Sistem

Analisis sistem merupakan tahapan awal dalam pengembangan perangkat lunak yang bertujuan untuk mengidentifikasi kebutuhan pengguna, menentukan fungsi yang harus disediakan sistem, serta merancang solusi yang sesuai dengan permasalahan yang ada. Pada proyek ini dikembangkan sebuah aplikasi pemesanan makanan sederhana berbasis Object-Oriented Programming (OOP) menggunakan bahasa pemrograman Python.

Aplikasi ini dirancang untuk membantu pengguna dalam melakukan pemesanan makanan secara mudah dan terstruktur. Sistem memungkinkan pengguna melihat daftar menu, memilih makanan, menambahkan pesanan ke keranjang, menghitung total harga, melakukan pembayaran, serta mencetak informasi pesanan. Melalui analisis kebutuhan ini, pengembang dapat memastikan bahwa sistem yang dibangun sesuai dengan kebutuhan pengguna dan memenuhi tujuan pengembangan perangkat lunak.

2.1.1 Analisis Kebutuhan Fungsional

Kebutuhan fungsional merupakan fungsi yang harus disediakan oleh sistem agar dapat memenuhi kebutuhan pengguna.

a) Pengolaan Data Menu

Sistem harus mampu menampilkan daftar makanan dan minuman yang tersedia beserta harga masing-masing menu. Pengguna dapat melihat informasi menu sebelum melakukan pemesanan.

Fungsi yang disediakan:

- Menampilkan daftar menu makanan.
- Menampilkan harga setiap menu.
- Menyediakan informasi menu yang dapat dipilih pengguna.

b) Pemesanan Makanan

Sistem harus memungkinkan pengguna memilih menu yang diinginkan dan menentukan jumlah pesanan.

Fungsi yang disediakan:

- Memilih menu makanan.
- Menentukan jumlah pesanan.
- Menambahkan pesanan ke keranjang belanja.
- Menyimpan data pesanan sementara

c) Perhitungan Total Harga

Sistem harus dapat menghitung total biaya yang harus dibayarkan berdasarkan jumlah dan harga makanan yang dipilih.

Fungsi yang disediakan:

- Menghitung subtotal setiap menu.
- Menghitung total keseluruhan pesanan.
- Menampilkan rincian biaya kepada pengguna.

d) Proses Pembayaran

Sistem harus menyediakan proses pembayaran untuk menyelesaikan transaksi.

Fungsi yang disediakan:

- Memasukkan nominal pembayaran.
- Menghitung jumlah kembalian.
- Menampilkan status pembayaran berhasil atau gagal.

e) Pencetak Informasi Pesanan

Sistem harus mampu menampilkan ringkasan transaksi setelah pembayaran berhasil dilakukan.

Fungsi yang disediakan:

- Menampilkan detail pesanan.
- Menampilkan total pembayaran.
- Menampilkan jumlah uang yang dibayarkan.
- Menampilkan kembalian

2.1.2 Analisa Kebutuhan non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan yang berkaitan dengan kualitas sistem yang dikembangkan.

a) Kemudahan Penggunaan (Usability)

Aplikasi harus memiliki tampilan yang sederhana bagi pengguna.

b) Kinerja Sistem (Performance)

Aplikasi harus mampu memproses data pesanan dan menghitung total harga dengan cepat tanpa mengalami keterlambatan yang signifikan.

c) Keandalan Sistem (Reliability)

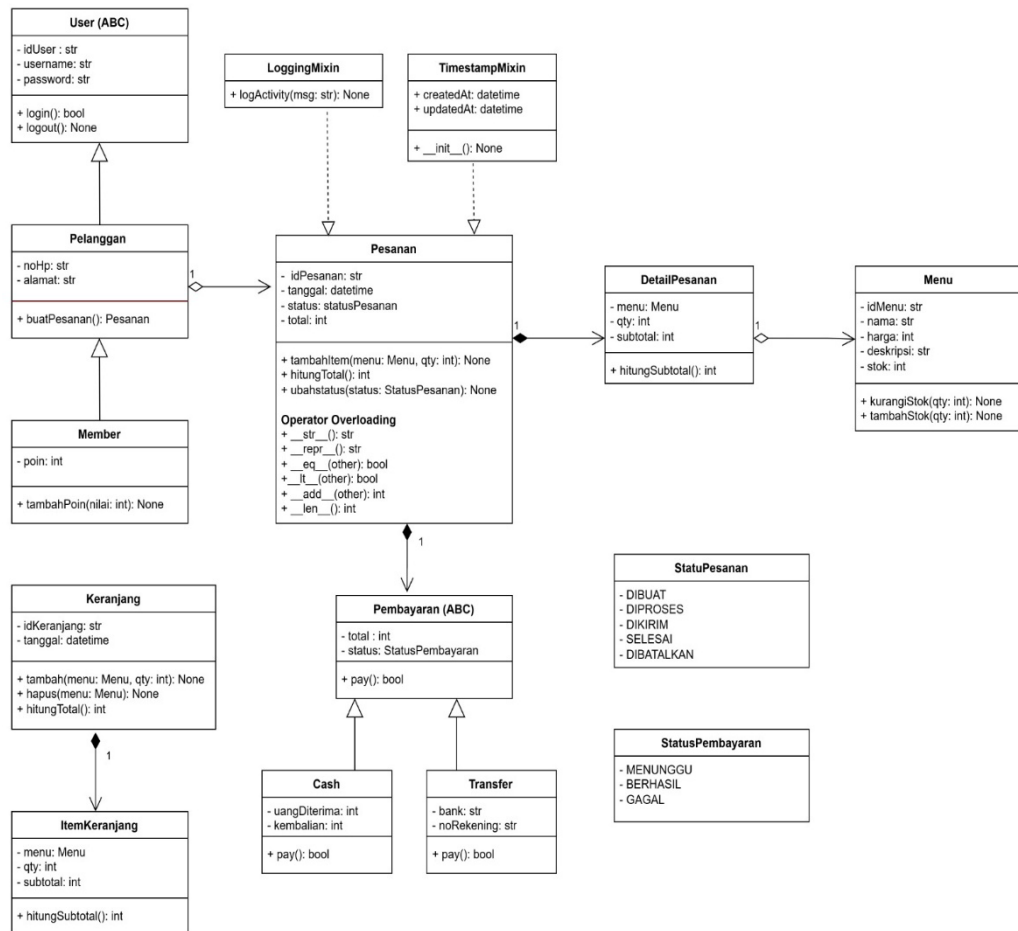
Aplikasi harus dapat berjalan dengan baik dan menghasilkan perhitungan yang akurat selama proses pemesanan maupun pembayaran.

d) Pemeliharaan Sistem (Maintainability)

Kode program harus disusun menggunakan konsep OOP sehingga mudah dikembangkan dan dipelihara pada masa mendatang.

2.2 Perancangan Sistem Menggunakan UML

Pada aplikasi pemesanan makanan sederhana ini agar mempermudah proses pengembangannya, maka sistem dirancang terlebih dahulu menggunakan Unified Modeling Language (UML). UML digunakan sebagai alat visualisasi untuk menggambarkan kebutuhan dan struktur sistem.



2.2.1 Use Case Diagram

Use Case Diagram digunakan untuk hubungan antara pengguna dengan fitur-fitur yang tersedia dalam sistem. Diagram ini menunjukkan aktivitas yang dapat dilakukan oleh pengguna serta bagaimana sistem merespons setiap aktivitas tersebut. Pada aplikasi pemesanan makanan sederhana terdapat dua aktor utama, yaitu Pelanggan dan Member. Member merupakan pengembangan dari Pelanggan yang memiliki fitur tambahan berupa pengumpulan poin setiap melakukan transaksi.

Fitur utama yang dapat dilakukan pengguna meliputi:

1. Login ke dalam sistem.
2. Melihat daftar menu makanan.
3. Menambahkan menu ke keranjang.
4. Mengelola isi keranjang.
5. Membuat pesanan.
6. Memilih metode pembayaran.
7. Melakukan pembayaran.
8. Melihat status pesanan.
9. Logout dari sistem.

Sementara itu, sistem bertugas mengelola data menu, menghitung total pembayaran, memperbarui status pesanan, mengurangi stok makanan, serta menyimpan riwayat transaksi pengguna.

2.2.2 Class Diagram

Class Diagram digunakan untuk menggambarkan struktur kelas, atribut, method, serta hubungan antar kelas dalam sistem. Diagram ini menjadi dasar penerapan konsep Object-Oriented Programming pada aplikasi. Pada aplikasi pemesanan makanan sederhana terdapat sebelas kelas utama, yaitu:

1. User (Abstract Base Class)

Kelas User merupakan kelas abstrak yang berfungsi sebagai induk dari seluruh pengguna sistem. Kelas ini memiliki atribut `idUser`, `username`, dan `password` serta method `login()` dan `logout()`.

2. Pelanggan

Kelas Pelanggan merupakan turunan dari User yang menyimpan informasi nomor telepon dan alamat pelanggan. Pelanggan memiliki kemampuan untuk membuat pesanan baru melalui method `buatPesanan()`.

3. Member

Member merupakan turunan dari Pelanggan yang memiliki atribut tambahan berupa poin loyalitas pelanggan. Poin akan bertambah setiap kali transaksi berhasil dilakukan.

4. Menu

Kelas Menu digunakan untuk menyimpan informasi makanan yang tersedia dalam sistem seperti nama menu, harga, deskripsi, dan stok. Kelas ini juga bertanggung jawab dalam proses penambahan maupun pengurangan stok.

5. Keranjang

Keranjang berfungsi sebagai tempat penyimpanan sementara menu yang dipilih pelanggan sebelum dibuat menjadi pesanan.

6. ItemKeranjang

Kelas ini menyimpan detail setiap menu yang dimasukkan ke dalam keranjang beserta jumlah dan subtotalnya.

7. Pesanan

Pesanan merupakan kelas utama yang mengelola transaksi pelanggan. Kelas ini menyimpan informasi tanggal pemesanan, status pesanan, total pembayaran, serta daftar item yang dipesan.

Selain itu, kelas Pesanan menerapkan operator overloading berupa:

- `str()`
- `repr()`
- `eq()`
- `lt()`
- `add()`
- `len()`

Operator tersebut digunakan untuk meningkatkan fleksibilitas objek saat dibandingkan, ditampilkan, maupun dihitung.

8. DetailPesanan

DetailPesanan digunakan untuk menyimpan rincian setiap menu yang terdapat dalam pesanan.

9. Pembayaran (Abstract Base Class)

Kelas abstrak yang menjadi dasar seluruh metode pembayaran dalam sistem. Status pembayaran yang digunakan terdiri dari:

- Menunggu
- Berhasil
- Gagal

10. Cash

Metode pembayaran tunai yang menghitung jumlah uang diterima dan kembalian pelanggan.

11. Transfer

Metode pembayaran melalui transfer bank yang menyimpan informasi nama bank dan nomor rekening tujuan.

Pada Class Diagram menunjukkan beberapa hubungan penting, yaitu:

- Pewarisan (*Inheritance*)



- Komposisi (*Composition*)

- Keranjang terdiri dari ItemKeranjang.
- Pesanan terdiri dari DetailPesanan.

- Agregasi (*Aggregation*)

- Pelanggan memiliki satu atau lebih Pesanan.

- Mixin

- LoggingMixin digunakan untuk mencatat aktivitas sistem.
- TimestampMixin digunakan untuk mencatat waktu pembuatan dan pembaruan data.

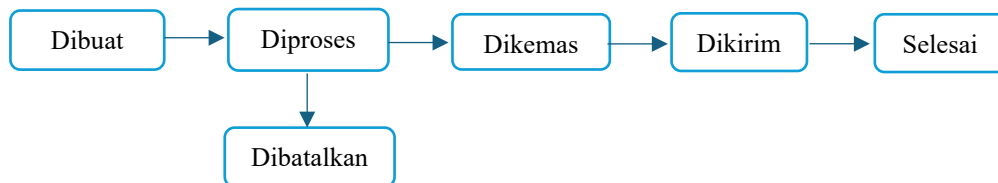
Melalui Class Diagram ini, seluruh kebutuhan sistem dapat diterjemahkan menjadi struktur program yang terorganisir.

2.2.3 Activity Diagram

Activity Diagram menggambarkan alur aktivitas yang dilakukan pengguna dari awal hingga akhir proses pemesanan makanan. Proses dimulai ketika pelanggan melakukan login ke dalam aplikasi. Setelah berhasil masuk, pelanggan dapat melihat daftar menu yang tersedia. Selanjutnya pelanggan memilih makanan dan menambahkannya ke dalam keranjang.

Setelah seluruh menu dipilih, pelanggan melakukan checkout sehingga sistem membentuk objek pesanan. Sistem kemudian menghitung total harga berdasarkan jumlah item yang dipilih. Pelanggan memilih metode pembayaran yang diinginkan, yaitu pembayaran tunai (Cash) atau transfer bank (Transfer). Setelah pembayaran dilakukan, sistem melakukan validasi transaksi.

Apabila pembayaran berhasil, status pembayaran berubah menjadi "Berhasil" dan sistem mulai memproses pesanan. Sebaliknya, apabila pembayaran gagal maka status pembayaran akan berubah menjadi "Gagal" dan pelanggan dapat melakukan pembayaran ulang. Setelah pembayaran berhasil, status pesanan akan mengalami perubahan sebagai berikut:



Pada kondisi tertentu, pesanan juga dapat berubah menjadi status "Dibatalkan". Pada proses tersebut dinamakan State Machine Diagram membantu pengembang memahami bagaimana suatu pesanan berpindah dari satu kondisi ke kondisi lainnya berdasarkan tindakan pengguna maupun proses yang dilakukan sistem

2.2.4 Sequence Diagram

Sequence Diagram digunakan untuk menggambarkan urutan komunikasi antar objek selama proses pemesanan makanan berlangsung.

Objek yang terlibat dalam proses ini meliputi:

- Pelanggan
- Keranjang

- Pesanan
- DetailPesanan
- Pembayaran
- Menu

Alur proses dimulai ketika pelanggan memilih menu dan menambahkannya ke keranjang. Keranjang kemudian menyimpan data item yang dipilih beserta jumlahnya. Ketika pelanggan melakukan checkout, sistem membuat objek Pesanan dan menyalin seluruh item dari keranjang ke DetailPesanan. Selanjutnya sistem menghitung total biaya yang harus dibayar. Setelah total biaya diperoleh, pelanggan memilih metode pembayaran. Objek Pembayaran kemudian memproses transaksi sesuai metode yang dipilih. Jika transaksi berhasil, status pembayaran berubah menjadi "Berhasil" dan sistem mengurangi stok menu yang telah dipesan. Pesanan kemudian diproses hingga mencapai status "Selesai". Seluruh aktivitas tersebut dicatat oleh LoggingMixin dan waktu transaksi disimpan menggunakan TimestampMixin. Dengan Sequence Diagram, pengembang dapat memahami urutan interaksi antar objek sehingga implementasi program menjadi lebih terstruktur dan sesuai dengan kebutuhan sistem.

2.3 Implementasi Konsep Object-Oriented Programming (OOP)

2.3.1 Encapsulation

Encapsulation atau enkapsulasi merupakan konsep yang digunakan untuk menggabungkan data dan fungsi yang berkaitan ke dalam satu kesatuan yang disebut kelas (*class*). Melalui konsep ini, data yang dimiliki suatu objek dapat dikelola secara lebih aman karena akses terhadap data dilakukan melalui method yang tersedia pada kelas tersebut.

Pada aplikasi pemesanan makanan sederhana, setiap kelas memiliki atribut dan method yang dirancang sesuai dengan tanggung jawabnya masing-masing. Sebagai contoh, kelas Menu menyimpan data berupa identitas menu, nama makanan, harga, deskripsi, dan jumlah stok. Pengelolaan stok tidak dilakukan secara langsung dari luar kelas, melainkan melalui method seperti `tambahStok()` dan `kurangiStok()`. Penerapan enkapsulasi juga terlihat pada kelas Pesanan yang bertanggung jawab mengelola data transaksi pelanggan.

Perhitungan total pesanan dilakukan melalui method `hitungTotal()` sehingga proses perhitungan tetap terkontrol dan tidak bergantung pada manipulasi data secara langsung dari luar objek.

2.3.2 Inheritance

Inheritance atau pewarisan merupakan mekanisme yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Konsep ini bertujuan untuk mengurangi pengulangan kode serta meningkatkan efisiensi dalam pengembangan perangkat lunak.

Pada aplikasi pemesanan makanan sederhana, pewarisan diterapkan pada kelas `User`, `Pelanggan`, dan `Member`. Kelas `User` berfungsi sebagai kelas induk yang menyediakan atribut umum seperti `idUser`, `username`, dan `password`. Kelas `Pelanggan` mewarisi seluruh karakteristik dari `User` kemudian menambahkan atribut nomor telepon dan alamat. Selanjutnya, kelas `Member` mewarisi seluruh karakteristik `Pelanggan` serta menambahkan fitur poin pelanggan.

User → Pelanggan → Member

Selain itu, pewarisan juga diterapkan pada sistem pembayaran. Kelas abstrak `Pembayaran` menjadi kelas induk yang menyediakan struktur dasar proses pembayaran. Kelas `Cash` dan `Transfer` kemudian mewarisi karakteristik tersebut dan mengimplementasikan mekanisme pembayaran sesuai dengan metode yang dipilih pelanggan.

2.3.3 Polymorphism

Polymorphism atau polimorfisme merupakan kemampuan objek untuk memiliki banyak bentuk perilaku melalui penggunaan method yang sama tetapi menghasilkan implementasi yang berbeda. Konsep ini memungkinkan sistem menjadi lebih fleksibel dalam menangani berbagai jenis objek.

Pada aplikasi pemesanan makanan sederhana, polimorfisme diterapkan pada proses pembayaran. Kelas `Pembayaran` mendefinisikan method `pay()` sebagai kerangka dasar proses pembayaran. Method tersebut kemudian diimplementasikan secara berbeda oleh kelas `Cash` dan `Transfer`.

Pada pembayaran tunai, method `pay()` digunakan untuk menghitung jumlah uang yang diterima dan kembalian pelanggan. Sementara itu, pada

pembayaran transfer, method pay() digunakan untuk memvalidasi data transfer berdasarkan informasi bank dan nomor rekening yang digunakan.

2.3.4 Abstraction

Abstraction atau abstraksi merupakan konsep yang digunakan untuk menyederhanakan kompleksitas sistem dengan menampilkan fitur-fitur penting dan menyembunyikan detail implementasi yang tidak perlu diketahui oleh pengguna. Dalam aplikasi pemesanan makanan sederhana, abstraksi diterapkan melalui penggunaan Abstract Base Class (ABC). Kelas User dan Pembayaran dirancang sebagai kelas abstrak yang tidak dapat digunakan secara langsung untuk membuat objek.

Kelas User hanya menyediakan struktur dasar yang harus dimiliki oleh seluruh jenis pengguna dalam sistem, seperti proses login dan logout. Implementasi lebih lanjut dilakukan oleh kelas turunannya, yaitu Pelanggan dan Member. Demikian pula pada kelas Pembayaran, sistem hanya mendefinisikan konsep umum pembayaran melalui method pay(). Detail implementasi pembayaran diserahkan kepada kelas Cash dan Transfer sesuai dengan karakteristik masing-masing metode pembayaran.

2.4 Pengujian Sistem

Pengujian sistem merupakan tahapan penting dalam pengembangan perangkat lunak yang bertujuan untuk memastikan bahwa seluruh fungsi yang telah dirancang dapat berjalan sesuai dengan kebutuhan pengguna. Pengujian sistem dilakukan untuk mencapai beberapa tujuan, yaitu:

No.	Fitur	Input	Hasil yang diharapkan
1.	Login	Username dan password benar	Berhasil masuk
2.	Login	Password salah	Login gagal
3.	Tambah Menu	Memilih menu	Menu masuk keranjang
4.	Checkout	Keranjang berisi item	Pesanan berhasil dibuat
5.	Pembayaran Cash	Uang cukup	Status pembayaran berhasil
6.	Pembayaran Transfer	Data transfer valid	Status pembayaran berhasil
7.	Update Status	Pesanan diproses	Status berubah sesuai alur

Berdasarkan proses pengujian yang dilakukan, sistem dinyatakan berhasil apabila seluruh fitur mampu menghasilkan keluaran yang sesuai dengan kebutuhan pengguna dan tidak ditemukan kesalahan yang mengganggu proses utama aplikasi. Selain itu, seluruh kelas dan objek yang dibangun menggunakan konsep Object-Oriented Programming harus dapat berinteraksi dengan baik sesuai rancangan UML yang telah dibuat. Dengan demikian, pengujian sistem tidak hanya berfungsi untuk memastikan kebenaran fungsi aplikasi, tetapi juga menjadi sarana untuk mengevaluasi kualitas implementasi perangkat lunak secara keseluruhan sehingga aplikasi pemesanan makanan sederhana dapat digunakan dengan lebih andal dan efektif.

2.5 Penerapan dan Evaluasi Prinsip SOLID

Dalam pengembangan aplikasi pemesanan makanan sederhana, prinsip SOLID diterapkan untuk menghasilkan struktur program yang lebih terorganisir, mudah dipelihara, dan mudah dikembangkan. Penerapan prinsip ini membantu setiap kelas menjalankan tugasnya secara spesifik sehingga mengurangi kompleksitas sistem dan meningkatkan kualitas desain perangkat lunak.

Prinsip Single Responsibility Principle (SRP) diterapkan dengan memberikan satu tanggung jawab utama pada setiap kelas. Sebagai contoh, kelas Menu bertugas mengelola data makanan, kelas Keranjang mengelola item yang dipilih pengguna, sedangkan kelas Pesanan berfokus pada pengelolaan transaksi. Dengan pembagian tugas yang jelas, perubahan pada satu kelas tidak akan memengaruhi fungsi kelas lainnya.

Prinsip Open-Closed Principle (OCP) dan Liskov Substitution Principle (LSP) diterapkan melalui penggunaan pewarisan pada kelas User, Pelanggan, Member, serta Pembayaran, Cash, dan Transfer. Struktur ini memungkinkan sistem dikembangkan dengan menambahkan kelas baru tanpa harus mengubah kode yang sudah ada, serta memastikan bahwa objek turunan dapat digunakan sebagai pengganti objek induknya tanpa mengganggu jalannya sistem.

Selain itu, Interface Segregation Principle (ISP) diterapkan dengan memastikan setiap kelas hanya memiliki atribut dan method yang sesuai dengan kebutuhannya. Sementara itu, Dependency Inversion Principle (DIP) diterapkan melalui penggunaan kelas abstrak Pembayaran yang menjadi perantara antara

proses pesanan dan metode pembayaran yang digunakan. Dengan pendekatan ini, sistem tidak bergantung pada implementasi pembayaran tertentu sehingga lebih fleksibel terhadap pengembangan di masa mendatang.

Berdasarkan evaluasi yang dilakukan, penerapan prinsip SOLID pada aplikasi pemesanan makanan sederhana telah membantu menciptakan sistem yang lebih modular, mudah dipahami, dan lebih siap untuk dikembangkan. Penerapan prinsip-prinsip tersebut juga mendukung penggunaan konsep Object-Oriented Programming secara optimal sehingga menghasilkan perangkat lunak yang lebih berkualitas dan terstruktur.

BAB III

IMPLEMENTASI DAN HASIL

Tahap implementasi aplikasi pemesanan makanan sederhana dilakukan menggunakan bahasa pemrograman Python dengan pendekatan Object-Oriented Programming (OOP). Proses pengembangan sistem dilakukan menggunakan Visual Studio Code sebagai lingkungan pengembangan (Integrated Development Environment/IDE) yang menyediakan berbagai fitur untuk mempermudah penulisan, pengujian, dan perbaikan kode program. Selain itu, sistem dijalankan pada sistem operasi Windows dan memanfaatkan beberapa pustaka bawaan Python seperti abc untuk penerapan Abstract Base Class, datetime untuk pengelolaan waktu transaksi, serta enum untuk mendefinisikan status pesanan dan pembayaran. Lingkungan pengembangan tersebut dipilih karena mendukung proses pembuatan aplikasi secara efektif dan mudah digunakan dalam implementasi konsep OOP.

3.1 Implementasi Struktur Kelas

Struktur kelas pada aplikasi dibangun berdasarkan rancangan UML yang telah dibuat. Setiap kelas memiliki tugas yang berbeda, seperti pengelolaan pengguna, menu makanan, keranjang, pesanan, dan pembayaran. Hubungan antar kelas menerapkan konsep pewarisan, komposisi, agregasi, dan abstraksi untuk menghasilkan sistem yang lebih terstruktur.

3.2 Implementasi Fitur Utama

Fitur utama yang diimplementasikan meliputi proses login, pengelolaan menu makanan, penambahan item ke keranjang, pembuatan pesanan, pemilihan metode pembayaran, serta pengelolaan status pesanan. Seluruh fitur dirancang agar dapat mendukung proses pemesanan makanan secara sederhana dan mudah digunakan.

3.3 Implementasi State Machine

State Machine diterapkan untuk mengatur perubahan status pesanan secara berurutan, mulai dari Dibuat, Diproses, Dikemas, Dikirim, hingga Selesai. Selain itu, sistem juga menyediakan status Dibatalkan untuk menangani kondisi tertentu. Pada proses pembayaran digunakan status Menunggu, Berhasil, dan Gagal.

3.4 Hasil dan Analisis Hasil

Pengujian dilakukan untuk memastikan setiap fitur berjalan sesuai dengan kebutuhan pengguna. Pengujian mencakup proses login, pengelolaan keranjang, pembuatan pesanan, pembayaran, dan perubahan status pesanan. Hasil pengujian menunjukkan bahwa seluruh fungsi utama dapat berjalan dengan baik.

Berdasarkan implementasi dan pengujian yang telah dilakukan, aplikasi mampu memenuhi kebutuhan dasar pemesanan makanan. Konsep OOP, UML, dan prinsip SOLID yang diterapkan membantu menghasilkan sistem yang lebih terorganisir, mudah dipahami, dan mudah dikembangkan pada tahap selanjutnya.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan penjelasan yang telah dipaparkan dalam makalah ini, adapun kesimpulan yang dapat diambil, antara lain:

1. Desain sistem aplikasi pemesanan makanan berhasil dirancang menggunakan diagram UML yang terdiri dari Use Case Diagram, Class Diagram, Activity Diagram, Sequence Diagram, dan State Machine Diagram sehingga struktur dan alur sistem dapat digambarkan dengan jelas.
2. Aplikasi berhasil diimplementasikan menggunakan konsep Object-Oriented Programming (OOP) dengan menerapkan prinsip Encapsulation, Inheritance, Polymorphism, dan Abstraction pada kelas-kelas yang terdapat dalam sistem.
3. Pengujian sistem berhasil dilakukan untuk memastikan setiap fitur, seperti login, pengelolaan menu, keranjang, pemesanan, pembayaran, dan perubahan status pesanan dapat berjalan sesuai dengan kebutuhan pengguna.
4. Prinsip-prinsip SOLID berhasil diterapkan pada desain dan implementasi aplikasi sehingga menghasilkan struktur program yang lebih modular, mudah dipelihara, serta mudah dikembangkan untuk penambahan fitur di masa mendatang.

4.2 Saran

Sebagai mahasiswa, penulis menyadari bahwa aplikasi pemesanan makanan sederhana yang dikembangkan masih memiliki berbagai keterbatasan. Oleh karena itu, diharapkan pada pengembangan selanjutnya aplikasi ini dapat dilengkapi dengan fitur yang lebih lengkap dan lebih kompleks. Penulis juga berharap proyek ini dapat menjadi sarana pembelajaran untuk memahami penerapan UML, Object-Oriented Programming (OOP),